HOW THIS SPECIFIC RAG SYSTEM WORKS

A detailed guide to the Enterprise Hybrid RAG architecture, structured querying, semantic retrieval, CRAG-style correction, and Self-RAG-style reflection

Dataset	Synthetic flight records (CSV; schema verified by the project's inspection script)
Backend	Python + FastAPI
Structured data	SQLite with bounded, read-only analytical queries
Semantic search	TF-IDF retrieval index
Corrective loop	CRAG-style relevance grading and bounded retries
Answer review	Self-RAG-style relevance/support checks and abstention

Purpose of this document
Explain what the project actually does, how a question moves through the system, what CRAG and Self-RAG mean in this implementation, how the parts fit together, how to run and test it, and which limitations must be addressed before enterprise deployment.

Important accuracy note: the current project implements CRAG-style and Self-RAG-style behavior using explicit application logic and lightweight heuristics. It is not a reproduction of the original research models or a guarantee that generated answers are correct.

Contents

1. Executive summary
2. What RAG means
3. The specific system and its components
4. The full flowchart
5. Data ingestion and indexing
6. Query routing: SQL, semantic, or hybrid
7. Structured SQL path
8. Semantic retrieval path
9. CRAG-style corrective retrieval
10. Answer construction and Self-RAG-style reflection
11. Citations, provenance, and abstention
12. Worked examples using the flight dataset
13. API and user interface
14. Security and enterprise considerations
15. Testing and evaluation
16. Run the project on Windows
17. Current limitations and roadmap
18. Glossary

1. Executive summary

This application is a hybrid question-answering system. It is called hybrid because it does not use a single search technique for every question. Instead, a router decides whether a question should be answered with structured database calculations, contextual search, or both.

The supplied flight dataset contains structured fields such as flight ID, airline, origin and destination airports, departure and arrival times, duration, altitude, speed, coordinates, status, and date. Exact questions about counts, rates, averages, or rankings are usually best answered by SQL. Questions that need a record to be found from descriptive wording can use semantic retrieval. Questions that need both can combine the two.

After retrieval, the system applies CRAG-style checks to assess whether the evidence appears relevant and may retry a weak search using a rewritten query. The answer is then assessed with Self-RAG-style checks: does it answer the question, and is it supported by the evidence? The system should return an abstention when it cannot support an answer rather than inventing facts.

The current implementation is a useful working foundation and learning system, not a fully certified enterprise platform. In particular, its default retrieval evaluation is lexical/heuristic, its contradiction handling is limited, and enterprise identity, role-based access control, tenant isolation, production load testing, and formal security assessment remain future work.

2. What RAG means

Retrieval-Augmented Generation (RAG) is a pattern in which an application retrieves relevant information from a data source and gives that evidence to a language model or answer-building component before producing a response. The intention is to ground the response in available information rather than relying only on what a model learned during training.

A simple analogy: imagine an open-book exam. Retrieval is finding the right pages; generation is writing the answer; citation checking is making sure the answer really follows from those pages. A RAG system can still fail if it retrieves the wrong pages, misreads them, or makes claims not supported by them. That is why retrieval evaluation and answer verification matter.

RAG is not the same as model fine-tuning. Fine-tuning changes model behavior by training; RAG generally leaves the model unchanged and supplies relevant context at query time. It is also not automatically a knowledge base with perfect recall: the quality depends on source quality, indexing, retrieval, query interpretation, and verification.

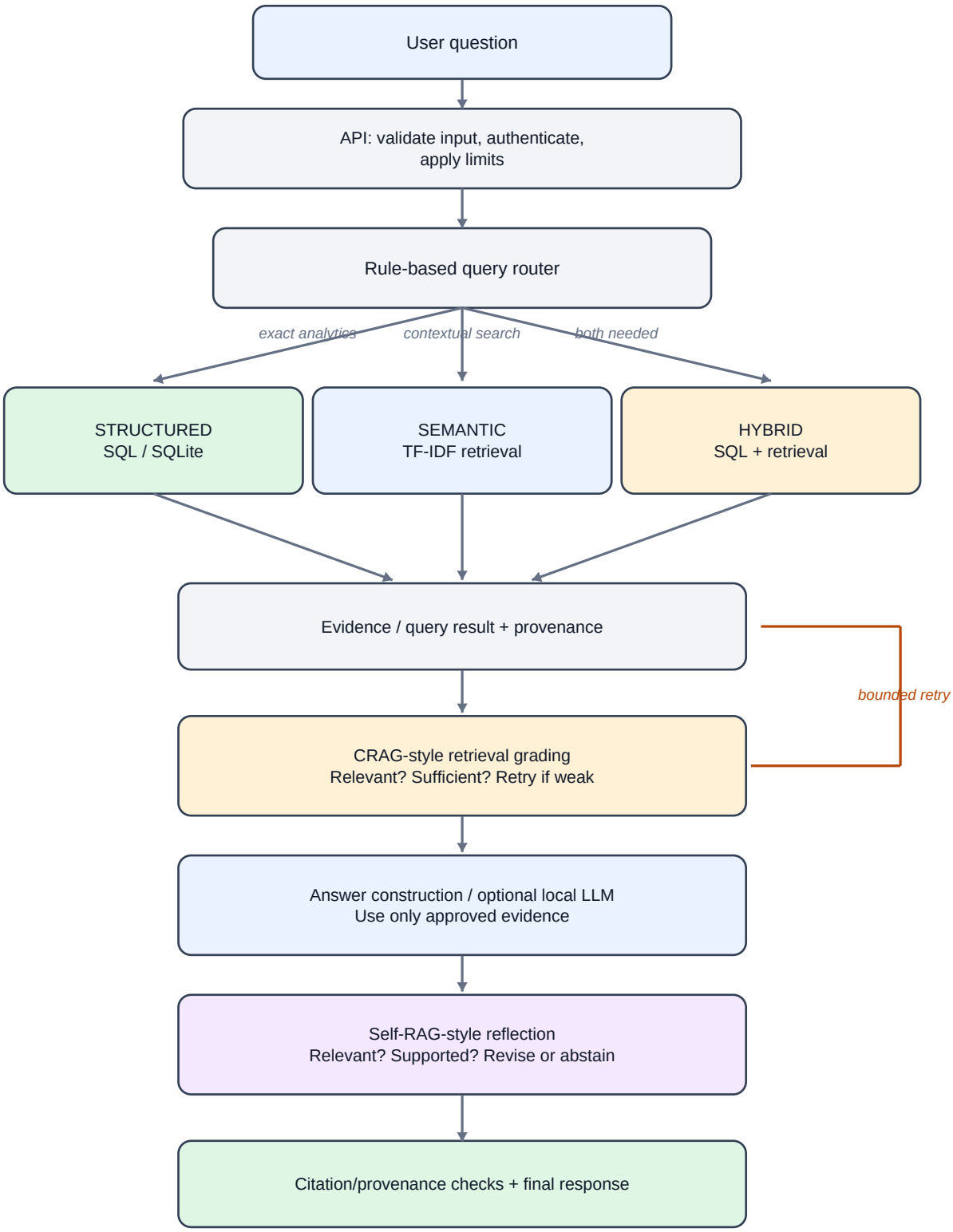
3. The specific system and its components

Component	Role in this project	Key caveat
FastAPI layer	Receives requests, validates inputs, applies configured API-key protection, and returns structured responses.	Optional API key is a baseline, not enterprise SSO or full RBAC.
Query router	Uses transparent rules to classify questions into structured, semantic, or hybrid routes.	Rule-based routing can misclassify ambiguous wording.
SQLite	Stores ingested flight rows and supports deterministic analytical queries.	Queries must remain bounded and read-only.
TF-IDF index	Finds records/documents with useful lexical overlap with the question.	It is not a modern semantic embedding model; synonyms may be missed.
CRAG-style retriever	Grades candidate relevance and retries with bounded query rewriting when evidence looks weak.	Lexical grading is a proxy, not a calibrated relevance model.
Answer builder / optional Ollama	Constructs a response from results; a local model may be used for supported semantic responses.	Model availability and factuality are not guaranteed.

Component	Role in this project	Key caveat
Self-RAG-style evaluator	Checks answer relevance/support and can trigger revision or abstention.	It does not implement the original trained Self-RAG reflection-token architecture.
Citations/provenance	Returns source identifiers and query provenance so a user can trace where a result came from.	A citation is useful only if it truly maps to the evidence used.

4. Full flowchart

This diagram shows the normal end-to-end path. The router chooses the retrieval path; the paths converge before retrieval-quality checks and answer verification. The retry loop is bounded to prevent endless repeated searches.



If evidence remains insufficient: return a clear abstention, not a guess.

5. Data ingestion and indexing

Before a question can be answered, the source data must be prepared. In this project the original CSV is the source of truth. The ingestion workflow validates and reads the data, copies records into SQLite, and turns records into searchable text for the TF-IDF index. The original CSV should not be silently overwritten.

Step 1 — Inspect. Check the archive and CSV schema, column names, types, missing values, duplicate rows, repeated flight IDs, and suspicious values. The report should distinguish data-quality warnings from definite errors; for example, missing altitude may be expected in some records and should not automatically invalidate the whole flight row.

Step 2 — Store structured records. SQLite is used to support exact filtering and aggregation. Database storage allows SQL to count rows, group by airline, filter by status, and calculate rates without asking a language model to do arithmetic from text fragments.

Step 3 — Build searchable text. A record can be serialized into a compact text description such as: “Flight FL123, airline Example Air, origin AAA, destination BBB, status cancelled, date 2026-01-01.” The original fields and stable identifiers should be preserved as metadata.

Step 4 — Index text. TF-IDF converts words into weighted numerical features. Common words carry less distinguishing value; words that are more specific to a record can carry more. Search compares the query representation with indexed records and returns candidates with the strongest lexical match.

Step 5 — Keep the index consistent. If the CSV changes, the database and search index must be refreshed. For a larger deployment, use content hashes, versioned indexes, incremental ingestion, and explicit deletion/update handling.

Why both SQLite and TF-IDF? SQLite is good at exact, repeatable calculations. TF-IDF is good at finding records based on words. They answer different classes of questions, so using both is more dependable than forcing one to do everything.

6. Query routing: SQL, semantic, or hybrid

The router is a decision point. It uses transparent rules to infer what kind of answer the question needs. A question asking “How many flights are cancelled?” strongly suggests an aggregate SQL query. “Find the flight going from Delhi to Mumbai around noon” may require filtering and searching record descriptions. “Which airline has the most cancellations, and what records support the result?” can require SQL plus source-level evidence.

The routing choice is not a guarantee. A question can be ambiguous, or a phrase can be interpreted incorrectly. A robust system should expose its selected route in safe metadata and allow tests to measure route accuracy. If the router is uncertain, the system should use a conservative fallback or ask a clarifying question rather than silently guessing.

Structured route: exact calculations and filters. Semantic route: find contextual evidence. Hybrid route: calculate with SQL and retrieve supporting records or context. Each route must preserve provenance so the final answer can be traced back to a source.

7. Structured SQL path

When the router selects a structured query, the application uses SQLite to calculate an answer. This avoids asking a language model to estimate counts or rates. For example, cancellation rate should be computed as cancelled records divided by the total records in the chosen population, multiplied by 100.

Cancellation rate (%) = (number of cancelled flights / number of flights in the denominator) × 100

The denominator must be clearly defined. Is it all records, only records with known status, or flights in a date range? A rate without its denominator and filters can be misleading. The application should report the scope used for calculations.

SQL safety is essential. The application should allow only the intended read-only query operations, limit returned rows, validate any generated query, and avoid unrestricted execution of model-written SQL. A prompt saying “do not execute dangerous SQL” is not a security boundary; enforcement must happen in code and database permissions.

Example analytical questions include counts by airline, most common destination, cancellation rates, average duration, date-filtered summaries, and flights matching a known flight ID. Each query should be tested against an independently computed expected result.

8. Semantic retrieval path

Semantic retrieval in this version is implemented with TF-IDF, a lexical information-retrieval technique. It is useful for matching words and terms in a query to indexed text. It does not understand meaning in the same way a strong transformer embedding model may. For example, “cancelled” and “called off” may not match well unless the query is rewritten or the vocabulary overlaps.

A typical path is: normalize the query; convert it into the same TF-IDF feature space as the indexed records; score candidate records; sort them by score; select a bounded number; and preserve each record’s source identifier and metadata. Low scores or weak overlap should be treated as a warning, not as proof that a record is irrelevant.

Potential improvements include adding BM25, a multilingual or domain-specific embedding model, a vector database when the data scale justifies it, metadata filtering, and a reranker. These should be introduced only with a benchmark demonstrating improvement, not simply because they are fashionable.

9. CRAG-style corrective retrieval

Corrective RAG (CRAG) is intended to catch cases where the first retrieval attempt is poor and correct the search before generating an answer. In this project, the “CRAG-style” mechanism grades candidate evidence using lightweight lexical/heuristic checks. If the evidence looks weak, it can rewrite the query and retry within a configured limit.

Conceptual process: retrieve candidates → grade relevance → if adequate, continue; if weak, rewrite or broaden the search → retrieve again → grade again → if still inadequate, abstain or report insufficient evidence.

The correction loop must be bounded. Without a maximum number of attempts, the system might repeat similar searches indefinitely, waste time, or produce an unstable answer. Keep a count of attempts and make the final response transparent when evidence remains insufficient.

A lexical grade is not a calibrated probability that the evidence is correct. It can be fooled by word overlap: a record can contain the right words but answer the wrong question. A future enterprise version should compare multiple grading methods against a labelled dataset, calibrate thresholds, and monitor false-positive and false-negative retrieval decisions.

10. Answer construction and Self-RAG-style reflection

After retrieval, the system constructs an answer using the available evidence. Depending on the route, this can be deterministic formatting of a SQL result or a generated response based on retrieved records. An optional Ollama integration can provide local model inference if the correct model is installed and configured.

Self-RAG, in the broad sense used here, adds reflection checks around answer generation. The application evaluates whether the response answers the question and whether important claims are supported by the retrieved evidence. If the checks fail, the response can be revised, qualified, or withheld.

> This implementation should be described as Self-RAG-style rather than the original Self-RAG research model. The original research approach includes specially trained reflection tokens and a particular model-training/inference design. This project approximates parts of the workflow through explicit application logic and heuristics.

A good evaluator asks claim-level questions: “Which source supports this sentence?”, “Does the cited record contain the stated value?”, “Did the answer change the denominator?”, and “Does the answer make a causal claim

that the data cannot establish?” A single vague confidence score is not enough.

The evaluator itself can be wrong. Therefore, use deterministic validation for arithmetic, dates, schema checks, permissions, and citations whenever possible. Use model-based judgments only where they add value, and measure their quality against labelled examples.

11. Citations, provenance, and abstention

Provenance means keeping track of where a piece of evidence came from. In a structured answer, provenance can include the query name, filters, and the dataset or database used. In semantic retrieval, it should include a stable source or flight identifier and any relevant metadata.

Citation checking should confirm that the cited source exists and that the final claim is actually supported by it. A citation that merely points to a related record is not enough. If the application cannot verify a claim, it should remove or qualify it, retrieve more evidence, or abstain.

Abstention is a correct behavior when the evidence is insufficient. A safe answer might say: “I could not verify that from the supplied dataset.” It is better than inventing a plausible-sounding answer. Abstention quality should also be tested: the system should answer supported questions and decline unsupported ones.

12. Worked examples using the flight dataset

Example A — “How many flights are cancelled?”

1. The router identifies a structured count request. 2. SQLite filters rows with the relevant status value. 3. The result is computed deterministically. 4. The application returns the count and provenance. 5. The answer is checked for consistency with the query result. A language model is not needed to do the arithmetic.

Example B — “Which airline has the highest cancellation rate?”

1. The router selects structured aggregation. 2. The SQL groups flights by airline. 3. It counts cancellations and the denominator for each airline. 4. It calculates each rate and ranks them. 5. The answer reports the winning airline, percentage, numerator, denominator, and scope. 6. The result should be independently tested against a separate aggregation implementation.

Example C — “Find the flight going from AAA to BBB around noon.”

1. The router recognizes a record-finding task. 2. The system filters or retrieves records using origin, destination, and time-related fields. 3. Candidate records are scored. 4. CRAG-style checks whether the candidates actually match the request. 5. If weak, it retries with a bounded rewrite. 6. The final result shows the matching flight IDs and source fields. If the time zone or date is missing and matters, ask for clarification.

Example D — “Explain why this airline has a high cancellation rate.”

The dataset may show the rate, but it may not contain causes. The system can report the observed statistic and explain limitations, but it must not infer weather, staffing, maintenance, or operational causes unless those are present in verified evidence. This is a classic case where the correct answer may include a limitation rather than a causal explanation.

13. API and user interface

FastAPI exposes the application over HTTP. The browser interface sends a question to the API and displays the response. The API is the boundary where input validation, authentication checks, rate limits (if configured), exception handling, and response schemas should be enforced.

The project’s API documentation can be opened at /docs when the local server is running. A health endpoint helps check whether the process is alive, but health is not the same as readiness: a service can be running while its data store or model provider is unavailable. For production, define separate readiness checks and avoid exposing sensitive configuration through diagnostics.

The interface should show the answer, route used, citations, and a clear message when evidence is insufficient. It should not reveal API keys, internal prompts, sensitive records the user is not authorized to see, or unrestricted database details.

14. Security and enterprise considerations

The supplied security notes correctly treat the current controls as a baseline. Before shared or production use, require a strong unique API key at minimum and preferably integrate with an enterprise identity provider and short-lived tokens. Add role-based authorization and tenant isolation if different users or departments must see different data.

Do not expose the development server, SQLite file, or local model server directly to the public internet. Use HTTPS behind a properly configured reverse proxy. Keep secrets in a secret manager, narrow CORS settings, limit request sizes and rates, and redact sensitive information from logs.

Prompt injection can be present in a user question or inside a retrieved document. Retrieved text must be treated as untrusted content, never as an instruction to override system policy or invoke tools. The model should not receive unrestricted shell, filesystem, network, or database access. Enforce these restrictions in the application, not just in a prompt.

Before enterprise deployment, add dependency scanning, threat-model review, backups, retention/deletion controls, audit logs, monitoring, incident response procedures, and deployment-specific privacy and data-residency review. Do not claim regulatory compliance without a formal assessment.

15. Testing and evaluation

Unit tests check small components in isolation. Integration tests check the whole route from request to response. Ground-truth evaluations check whether the answer is correct against independently calculated or labelled results. Security tests attempt unauthorized access, SQL injection, prompt injection, malformed requests, and resource exhaustion.

Useful metrics include retrieval precision/recall, ranking quality, citation correctness, claim-support rate, answer accuracy, abstention precision/recall, correction success rate, latency, model token usage, and error rate. Every metric needs a definition and a reproducible dataset. A passing test suite demonstrates only the cases that were tested; it does not prove there are no defects.

The project’s own evaluation report should be treated as the source for the actual test commands and observed results. Do not assume a test passed just because a test file exists. Record skipped tests, missing services, and untested providers explicitly.

16. Run the project on Windows

From PowerShell, change into the project directory. You do not need to activate the virtual environment if Windows blocks Activate.ps1; call the virtual environment’s Python executable directly.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[dev]"
.\.venv\Scripts\python.exe scripts\inspect_data.py
.\.venv\Scripts\python.exe scripts\ingest_data.py
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe scripts\run_evaluation.py
.\.venv\Scripts\python.exe -m uvicorn enterprise_rag.main:app --reload
```

Then open http://127.0.0.1:8000 for the application or http://127.0.0.1:8000/docs for the interactive API documentation. Exact commands may need adjustment if the repository package layout or entry point has changed; use the project README and inspect the actual files if a command fails.

17. Current limitations and roadmap

Area	Current state / recommended next step
Retrieval quality	TF-IDF is lexical and can miss synonyms or paraphrases. Benchmark a BM25 or embedding-based alternative before replacing it.
CRAG evaluation	Current grading is heuristic. Calibrate it against labelled relevant/irrelevant examples and measure correction success.
Self-RAG evaluation	The current logic is an approximation. Add claim-level support checks and stronger contradiction detection, with tests.
Model providers	Ollama requires a locally installed model and service. Gemini-specific integration should not be considered available until implemented and tested.
Security	Optional API-key protection is not SSO, fine-grained RBAC, or tenant isolation.
Operations	Add production logging/metrics, rate limiting, backup and restore, deployment automation, load testing, and security scanning.
Data scale	SQLite and an in-memory TF-IDF index suit a modest dataset; larger workloads may require a managed database, persistent search index, and measured capacity planning.

18. Glossary

API: A defined way for software systems to communicate.

Abstention: A deliberate decision not to answer because evidence is insufficient.

Chunk: A smaller piece of a document or record prepared for retrieval.

Citation / provenance: Information identifying the source behind a claim or result.

CRAG: Corrective Retrieval-Augmented Generation; a pattern that evaluates and attempts to improve weak retrieval.

Embedding: A numeric representation of text used to compare semantic similarity.

FastAPI: A Python framework for building HTTP APIs.

Ground truth: An independently verified expected answer used to evaluate a system.

Hybrid retrieval: Combining structured database querying with document or semantic search.

Index: A data structure that helps find relevant records efficiently.

Prompt injection: Malicious or conflicting instructions supplied in user input or retrieved content.

RAG: Retrieval-Augmented Generation.

Recall: The fraction of all relevant items that a retrieval system successfully finds.

Precision: The fraction of retrieved items that are actually relevant.

Self-RAG: A research approach that adds retrieval and answer reflection; this project implements a simplified Self-RAG-style workflow.

SQL: Structured Query Language, used to filter and aggregate relational data.

TF-IDF: Term Frequency–Inverse Document Frequency, a method for weighting terms in text retrieval.

Vector database: A database designed to store and search numeric vector representations.

Final takeaway

This system combines deterministic data analysis with contextual retrieval, then adds two quality-control ideas: CRAG-style retrieval correction and Self-RAG-style answer reflection. Its value depends on preserving the distinction between what the data proves, what the system infers, and what remains unknown. For enterprise use, strengthen and measure the evaluators, enforce access controls in code, test against ground truth, and document every limitation honestly.